

Infotact Technical Internship Program: Software Engineering Project Specifications and Evaluation Criteria

The following document establishes the technical requirements, architectural directives, and sprint-based development roadmaps for three distinct software engineering projects. Designed for execution within a fast-paced technology environment in Bengaluru, Karnataka, these specifications serve as comprehensive blueprints for Java engineering interns entering the infotact startup ecosystem. The documentation provides structured frameworks intended to guide the development of scalable, cloud-native applications focused on the Logistics, Transport, and Supply Chain niches.

The primary directive of this documentation is to standardize the enterprise Java engineering process, ensuring that all deliverables align with modern Spring Boot microservices architecture, rigorous version control protocols, and robust security standards. Consequently, engineering interns must operate at a professional level, demonstrating proficiency in Object-Oriented Programming (OOP), RESTful API design, database normalization, and Continuous Integration.

This document outlines the requisite tasks, technological paradigms, and evaluation criteria for successful internship completion. It is imperative to note that the assessment of these projects relies heavily on the transparency and consistency of the development lifecycle. **The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.**

Project 1: Enterprise Warehouse Management System (WMS)

Executive Problem Statement

With the exponential boom in e-commerce and rapid delivery expectations, traditional warehouse operations relying on manual spreadsheets face severe operational bottlenecks. Fragmented systems lead to inaccurate inventory synchronization, misplaced stock, and inefficient picker routing, which ultimately results in costly stockouts and delayed order fulfillment.

The objective of this project is to architect a robust, cloud-based Warehouse Management System (WMS) utilizing Java and Spring Boot. The platform must automate core warehouse operations, including real-time inventory tracking, optimized receiving and putaway logic, and efficient order picking workflows. This system will serve as the digital backbone for mid-sized logistics companies transitioning to automated supply chain management.

Business Objectives and Key Performance Indicators

To ensure the solution delivers tangible enterprise value, the system must maintain absolute data integrity across all inventory transactions. The primary engineering objective is the realization of a real-time inventory engine capable of preventing race conditions during concurrent stock updates.

Success will be measured by the system's ability to maintain ACID-compliant database transactions. Key functionalities include generating and reading barcode/QR data for automated data capture, minimizing the manual data entry required by warehouse floor staff. The API response times must remain below 200 milliseconds to support fast-paced scanning operations on the warehouse floor.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Warehouse Manager	Macroscopic visibility of inventory levels, multi-warehouse tracking, and capacity optimization.	Uses the dashboard to monitor stock thresholds, manage supplier purchase orders, and track overall warehouse efficiency.
Floor Operator (Picker/Packer)	Fast, clear instructions on item locations and automated data capture.	Utilizes mobile-friendly interfaces to scan inbound items, execute system-directed putaway to specific storage bins, and pick items for outbound shipping.

Minimum Viable Product Specifications

The foundational feature is the Hierarchical Inventory Catalog. The database schema must handle complex relationships between Warehouses, Zones, Aisles, and specific Bins.

The core operational modules include the Receiving & Putaway Engine, which logs incoming shipments and assigns them to optimal storage locations based on available capacity. The system must also feature an Order Fulfillment API that changes an order's state dynamically (e.g., Pending $\rightarrow$ Picking $\rightarrow$ Packed $\rightarrow$ Shipped).

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend Framework	Java 17+, Spring Boot 3.x	Spring Boot simplifies the creation of production-ready, stand-alone Spring applications, providing built-in tools for enterprise scalability.
Data Access Layer	Spring Data JPA / Hibernate	Essential for managing complex relational database schemas and executing secure, parameterized queries to prevent SQL injection.
Primary Database	PostgreSQL	A highly robust open-source relational database required for enforcing strict foreign key constraints and transactional integrity in inventory ledgers.
Frontend Application	React.js	Provides a dynamic, component-based user interface that consumes the Spring Boot REST APIs efficiently.

Four-Week Engineering Roadmap

Week 1: Spring Boot Scaffolding and Relational Schema Design

Interns must initialize the GitHub repository and set up a Spring Boot project using Spring Initializr, importing dependencies for Web, JPA, and PostgreSQL. The primary task is designing the Entity-Relationship (ER) model. The intern must create JPA Entities for `Product`, `Warehouse`, `StorageBin`, and `InventoryItem`, meticulously defining `@OneToMany` and `@ManyToOne` relationships. The week concludes with developing foundational CRUD REST controllers.

Week 2: Transactional Inventory Logic and Putaway Algorithms

Week two focuses on business logic and data integrity. The intern must implement the Receiving service. This requires utilizing Spring's `@Transactional` annotation to ensure that when a new shipment arrives, the stock count increases and the system successfully maps the items to a specific `StorageBin` as a single atomic operation.

Week 3: Barcode Integration and Order Processing Workflow

The third week introduces operational features. The intern will integrate a Java library (such as ZXing) to automatically generate printable Barcode/QR code images for distinct product SKUs. Concurrently, the intern must develop the Order Management service, creating endpoints that decrement available stock when an order is moved to the "Packed" state, explicitly handling `InsufficientStockExceptions`.

Week 4: Security, Frontend Integration, and Testing

The final sprint requires securing the APIs using Spring Security and JSON Web Tokens (JWT), establishing role-based access for 'ADMIN' and 'OPERATOR' roles. The intern will connect a React frontend to the backend APIs, utilizing Axios or Fetch. Finally, unit tests must be written for the core inventory services using JUnit 5 and Mockito, ensuring the repository contains a continuous, daily commit history across the 4-week span.

Project 2: Fleet Management and Route Optimization Engine

Executive Problem Statement

Logistics and freight companies operate on razor-thin margins. Inefficient route planning, excessive fuel consumption, and a lack of real-time visibility into fleet locations significantly drain profitability. Dispatchers frequently rely on static maps or human intuition to assign deliveries, failing to account for dynamic constraints such as delivery time windows and vehicle capacities.

The objective of this project is to build a centralized Fleet Management REST API equipped with a Route Optimization engine. The application must handle vehicle assignments, track maintenance schedules, and algorithmically sequence multiple delivery stops to minimize total travel distance and fuel expenditure.

Business Objectives and Key Performance Indicators

The strategic goal is to reduce operational costs through algorithmic efficiency. The backend system must maintain an accurate registry of fleet assets, driver profiles, and daily delivery manifests.

Success will be quantified by the optimization engine's ability to take a list of disorganized waypoints and return a sequentially ordered route. The system must successfully integrate with external mapping services (e.g., Google Distance Matrix API or OpenStreetMap) to calculate the shortest paths.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Fleet Dispatcher	Complete visibility of vehicle availability, driver statuses, and route efficiency.	Uses the system to assign daily manifests to available trucks, track real-time mocked locations, and monitor vehicle maintenance alerts.
Delivery Driver	Clear, sequential delivery instructions and route mapping.	Interfaces with a simulated mobile endpoint to view their daily optimized stops and update the delivery status of individual packages.

Minimum Viable Product Specifications

The foundational feature is the Fleet Registry API, a highly structured set of endpoints managing entities for **Vehicle** (tracking capacity, license plate, maintenance status) and **Driver** (tracking license validity, shift hours).

The core feature is the Dispatch and Routing Engine. Given a payload of 5-10 delivery addresses for a specific truck, the Spring Boot application must make HTTP requests to an external Distance API, evaluate the travel times, and utilize a basic optimization algorithm (such as a greedy approach for the Traveling Salesperson Problem) to return the most efficient delivery sequence.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend API	Java, Spring Boot, Spring WebFlux	Spring WebFlux can be introduced for handling non-blocking, asynchronous streams of GPS location data efficiently.

Database	MySQL	A standard, highly reliable relational database for storing structured fleet and driver telematics data.
External Integration	RestTemplate / Spring Cloud OpenFeign	Required for making programmatic HTTP calls to external third-party routing and mapping APIs.

Four-Week Engineering Roadmap

Week 1: Fleet Schema Design and Database Initialization

The project initiates with the intern designing the relational schema in MySQL and configuring the Spring Boot application. Entities must be created for `Driver`, `Vehicle`, `DeliveryTask`, and `Route`. The intern will implement the Data Access Objects (DAOs) using Spring Data JPA and build the core endpoints to register new vehicles and assign drivers to them.

Week 2: External API Integration and Route Optimization Logic

Week two centers on algorithmic complexity. The intern must integrate a third-party mapping API (like OSRM or Google Maps Distance Matrix) using Spring's `RestTemplate` or `WebClient`. The intern will write a Service class that accepts a list of coordinate pairs, fetches the distance matrix from the external API, and calculates an optimized sequence of stops to minimize total travel time.

Week 3: Dispatch Workflow and State Management

The third week transitions to workflow management. The intern will build the Manifest generation logic, linking optimized routes to specific drivers and vehicles. A state machine logic must be implemented for deliveries (e.g., `UNASSIGNED`, `DISPATCHED`, `IN_TRANSIT`, `DELIVERED`). The intern must configure robust exception handling utilizing `@RestControllerAdvice` to manage API rate limits or invalid address inputs gracefully.

Week 4: Security, Documentation, and Containerization

The final week is dedicated to professional deployment standards. The intern must implement Swagger (Springdoc OpenAPI) to auto-generate interactive API documentation for the frontend/mobile teams. The application must be containerized by writing a `Dockerfile`. The intern will finalize the repository, ensuring all semantic commit messages clearly explain the logic behind the routing algorithms and architecture choices.

Project 3: Real-Time Shipment Tracking Portal & Logistics Marketplace

Executive Problem Statement

The freight and logistics industry suffers from severe transparency issues. Shippers frequently struggle to find reliable carriers with available capacity, while end-customers are left in the dark regarding the real-time geographic status of their high-value shipments.

This project requires the development of a dual-sided Logistics Marketplace combined with a Real-Time Tracking Portal. The platform must allow shippers to post freight loads, enable carriers to bid on them, and subsequently provide a WebSocket-driven tracking dashboard where customers can watch the simulated progress of their shipments on a map.

Business Objectives and Key Performance Indicators

The strategic vision is to act as a digital intermediary, replacing manual freight brokering with automated algorithmic matching.

From an engineering standpoint, the system's success relies on asynchronous event handling. The intern must successfully implement a message broker or WebSocket architecture to push live location updates from the server to the client without forcing the client to aggressively poll the database, thereby ensuring a highly scalable, real-time tracking experience.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Shipper	The ability to find carriers quickly and track cargo continuously.	Posts a new load detailing weight and destination, accepts carrier bids, and monitors transit status.
Carrier / Driver	Access to available jobs and an easy way to update transit status.	Browses a marketplace board of available shipments, submits a price bid, and simulates GPS ping updates once a job is won.

End-Customer	Accurate ETA notifications and geographic visibility.	Enters a tracking ID into a web portal to view live updates and delivery confirmation details.
---------------------	---	--

Minimum Viable Product Specifications

The application foundation is the Freight Bidding Engine. The intern must design a relational structure handling Users, Load Postings, and Bids, allowing a Shipper to officially "Award" a load to a specific Carrier.

The core technical deliverable is the Real-Time Telematics Module. The intern will build a mechanism where a simulated driver app sends periodic GPS coordinates (Latitude/Longitude) to the Spring Boot backend. The backend must immediately broadcast these coordinates to the specific shipper's frontend dashboard utilizing WebSockets and STOMP messaging protocols.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend Services	Java, Spring Boot	Provides the secure framework necessary for handling user authentication, bidding logic, and business rules.
Real-Time Messaging	Spring WebSockets (STOMP) / RabbitMQ	WebSockets provide full-duplex communication channels over a single TCP connection, essential for live tracking dots on a map.
Database	PostgreSQL	Handles the transactional bidding and order management data securely.

Four-Week Engineering Roadmap

Week 1: Marketplace Schema and Authentication

The project begins with establishing the Spring Boot repository. The intern must configure PostgreSQL and design the schema to support the multi-tenant marketplace: **Users** (distinguishing Shipper vs. Carrier roles), **Shipments**, and **Bids**. Spring Security with JWT must be implemented to ensure Carriers cannot access Shipper-specific data, and vice versa.

Week 2: Load Board API and Bidding Logic

Week two focuses on the core marketplace mechanics. The intern will develop REST APIs allowing Shippers to create a **Shipment** with origin, destination, and weight parameters. Carriers must be able to GET a filtered list of available shipments and POST a **Bid**. Complex logic must be written to ensure that once a Shipper accepts a bid, all other bids are rejected, and the Shipment state is locked to **AWAITING_PICKUP**.

Week 3: WebSocket Configuration and Live Tracking

The third week tackles real-time event streaming. The intern must configure Spring WebSockets. A dedicated endpoint must be created to receive mocked GPS coordinates from a simulated driver. Upon receiving a coordinate payload for a specific **Shipment ID**, the server must instantly broadcast that payload to any authenticated client currently subscribed to that specific shipment's WebSocket topic.

Week 4: Dashboard Integration, Testing, and Final Review

The final sprint involves wiring the backend systems to a React-based frontend dashboard. The frontend must plot the WebSocket coordinates on a map UI (e.g., using Leaflet or Google Maps React). The intern must ensure the connection gracefully handles network reconnects. The repository must be finalized with a comprehensive **README.md** detailing the WebSocket architecture, and the commit history must reflect a steady progression of features over the entire four weeks.

Universal Engineering and Version Control Guidelines

To simulate a rigorous, enterprise-grade development environment indicative of top-tier product companies in Bengaluru, strict adherence to modern software engineering practices is an absolute requirement. The core tenet of this internship program is continuous, transparent, and high-quality contribution. Engineering is not merely the final compiled **.jar** file; it is the iterative, documented process of problem-solving.

Mandatory Evaluation Protocol

The evaluation of the intern's performance and the final project submission is governed by a strict, non-negotiable directive regarding version control consistency.

The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

A project submitted with a compressed commit history—for example, uploading the entire codebase in a massive, monolithic push during the final week or on the final day—will result in immediate disqualification. Professional engineering is a continuous process of iteration, and the version control history must accurately reflect daily progress, architectural pivots, code refactoring, and debugging efforts.

GitHub Operational Standards and Security Practices

To pass the technical evaluation, interns must internalize and strictly adhere to the following Git and GitHub operational standards.

1. Commit Frequency, Granularity, and Semantic Messaging

Commits act as the immutable ledger of an engineer's thought process. Interns are expected to commit code frequently, breaking down complex features into logical, highly isolated units of work. A minimum of three to five meaningful commits per active development day is the expected baseline. Monolithic commits containing thousands of lines of code spanning multiple disparate features are strictly prohibited.

Commit messages must communicate the exact intent and context of the change. Interns must utilize structured prefixes such as `feat:`, `fix:`, `refactor:`, or `test:` to maintain a highly readable repository history (e.g., `feat: implemented JWT filter for carrier endpoints`).

2. Branching Strategy and Pull Request (PR) Etiquette

Direct commits to the `main` or `master` branch are entirely forbidden. All new development must occur on isolated feature branches. Code is merged into the main branch exclusively via Pull Requests. Each PR must contain a comprehensive description outlining the specific problem solved and the architectural approach taken.

3. Continuous Integration and Cryptographic Security (CI/CD)

To demonstrate proficiency in modern DevOps practices, repositories must utilize GitHub Actions. A basic pipeline must be established that automatically runs `mvn clean test` (or the Gradle equivalent) on every Pull Request, failing the build if any unit tests fail. Hardcoding API keys, database passwords, or secret JWT tokens within the Java source code or `application.properties` is grounds for immediate failure. All sensitive data must be injected via secure Environment Variables.

By strictly enforcing these comprehensive specifications, the program ensures that deliverables transcend mere functional prototypes, transforming into robust, scalable, and highly maintainable Java applications indicative of elite industry standards.